

Introduction & Hook

“If a VR scene contains 10,000 objects, does the GPU render all of them every frame?”

Explain:

Actually, the system renders **only objects visible to the camera**.

Objects that cannot be seen are **culled** (removed from rendering).

This saves:

- GPU power
 - Memory bandwidth
 - Frame time
-

PART 1: What is Culling?

Basic Concept of Culling

Definition

Culling is the process of removing objects from rendering that are not visible to the viewer.

Types of culling in graphics:

- Back-face culling
- View frustum culling
- Occlusion culling

In VR engines like Unity and Unreal, culling happens automatically.

Why Culling is Important

Without culling:

- All objects rendered
- GPU overload
- Frame rate drops

With culling:

- Only visible objects rendered
- Faster rendering
- Smooth VR experience

PART 2: View Frustum Culling

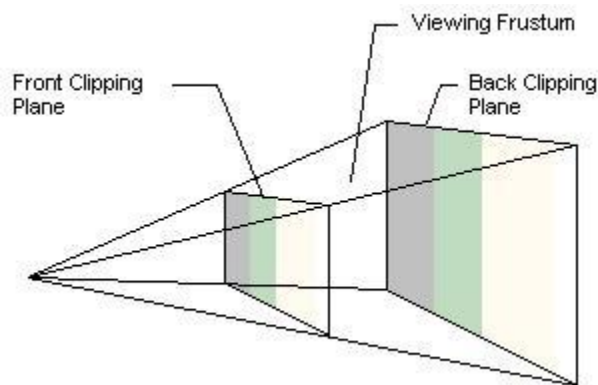
What is View Frustum?

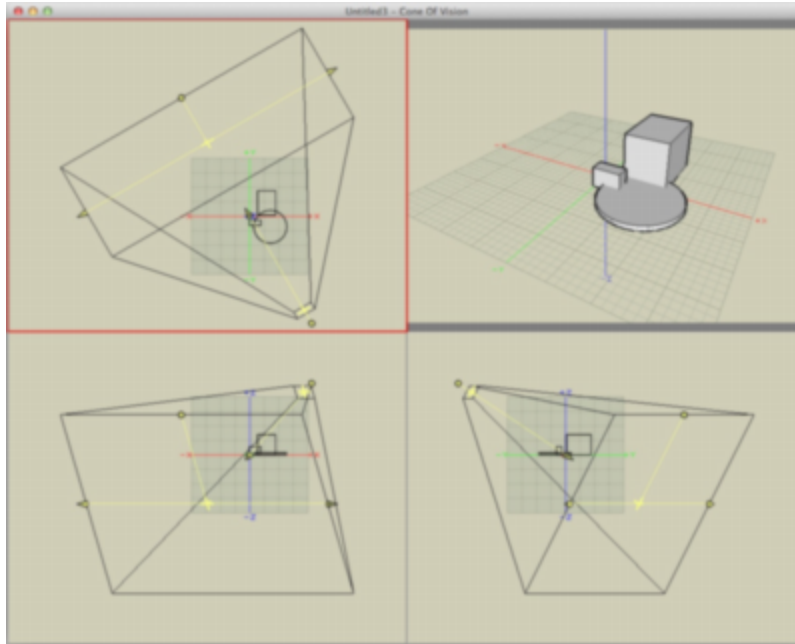
The **view frustum** is the 3D region visible to the camera.

It looks like a **truncated pyramid**.

The frustum is defined by:

- Near plane
- Far plane
- Left plane
- Right plane
- Top plane
- Bottom plane





4

How View Frustum Culling Works

Steps:

1. Camera defines viewing volume
 2. Engine checks each object's bounding box
 3. If object outside frustum → ignore rendering
 4. If object inside frustum → render normally
-

Example

In a VR city scene:

- Buildings behind the camera → culled
 - Buildings outside field of view → culled
 - Only visible buildings rendered
-

PART 3: Occlusion Culling

What is Occlusion?

Sometimes objects are **inside the view frustum but still not visible** because another object blocks them.

Example:

A building hides another building behind it.

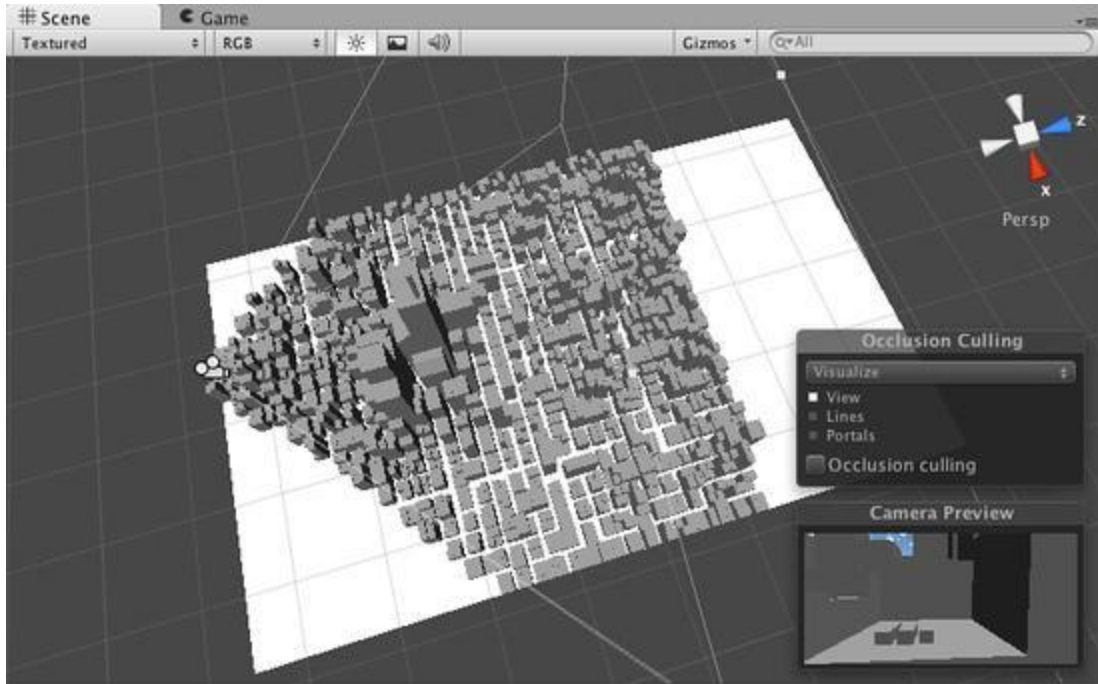
Even though the hidden building is inside the viewing volume, it cannot be seen.

Definition

Occlusion culling removes objects that are hidden behind other objects.

Objects that block others are called **occluders**.

Hidden objects are called **occludees**.



4

How Occlusion Culling Works

Steps:

1. Identify large objects (walls, terrain, buildings)
2. Treat them as occluders
3. Check objects behind them
4. Hidden objects are not rendered

Example

VR indoor scene:

Room wall blocks objects in next room.

Engine skips rendering those hidden objects.

Frustum vs Occlusion Culling

Feature	View Frustum Culling	Occlusion Culling
Based on	Camera viewing volume	Object visibility
Removes	Objects outside camera view	Objects hidden behind others
Complexity	Simple	More complex
Performance	Fast	More accurate

Explain important point:

Frustum culling happens first, then occlusion culling.

Real VR Example

In a VR game:

Player stands in a hallway.

- Rooms behind player → frustum culled
- Objects behind walls → occlusion culled

Result:

- GPU renders only necessary objects
- VR maintains high FPS

Recap + Questions

Key Points

- Culling removes invisible objects
- View frustum culling removes objects outside camera view
- Occlusion culling removes objects hidden behind others
- Both improve rendering performance

Quick Questions

- What is a view frustum?
- Why is frustum culling important?

- What is an occluder?
 - Which culling method removes hidden objects?
-

1.Numerical: View Frustum Culling (Object Visibility)

Problem

A camera view frustum has the following limits:

- Near plane = **1**
- Far plane = **10**

An object is located at position:

$(x,y,z)=(2,3,12)$

Determine whether the object will be rendered or culled.

Solution

Check Z value against frustum depth.

Object Z coordinate = **12**

Frustum range:

$$1 \leq z \leq 10$$

Since:

$$12 > 10$$

The object lies **outside the far plane**.

 **Result:** Object will be **culled**.

2. Numerical: Frustum Culling with Multiple Objects

Problem

A VR scene contains **6 objects** with the following Z coordinates:

Object Z Position

A	2
B	4
C	8
D	12
E	-3
F	6

Camera frustum limits:

Near plane = **1**

Far plane = **10**

Which objects will be rendered?

Solution

Valid range:

$$1 \leq z \leq 10$$

Check each object:

Object Z Result

A	2	Visible
B	4	Visible
C	8	Visible
D	12	Culled
E	-3	Culled
F	6	Visible

✓ **Rendered objects:** A, B, C, F

✗ **Culled objects:** D, E

3. Numerical: Performance Impact of Culling

Problem

A VR scene contains **1000 objects**.

After view frustum culling:

- **300 objects are outside the frustum**

Calculate how many objects the GPU needs to render.

Solution

Total objects:

1000

Objects culled:

300

Objects rendered:

$1000 - 300 = 700$

✅ **700 objects are rendered**

4. Numerical: Occlusion Culling

Problem

A VR city scene contains **20 buildings**.

A large building blocks **8 buildings behind it** from the camera view.

Determine how many buildings will be rendered.

Solution

Total buildings:

20

Occluded buildings:

8

Rendered buildings:

$20 - 8 = 12$

 **12 buildings are rendered**

5. Numerical: Combined Culling Example

Problem

A VR scene has **50 objects**.

- **15 objects are outside the view frustum**
- **10 objects are hidden behind walls**

How many objects are rendered?

Solution

Step 1: Frustum culling

$50 - 15 = 35$

Step 2: Occlusion culling

$35 - 10 = 25$

✓ Final rendered objects = 25